

KenaKata

A Marketplace Order & Settlement Engine

Object-Oriented Programming — Standalone Lab

Marks: 15 **Language:** Java (JDK 17) **Approach:** Build from scratch, test-driven

You are given this specification and a test file `KenaKataTest.java`. Submit your `.java` sources; do not modify the test file.

Problem Statement

KenaKata is an online marketplace. Independent sellers list many different kinds of product; a customer fills a basket with products and optional add-ons, applies a coupon, chooses a delivery zone and a way to pay, and checks out. The system must turn that basket into a fully priced order — correct to the taka — reserve stock and take payment atomically, and at day's end settle every seller for what they are owed. You will build this pricing-and-settlement engine from scratch. The catalogue mixes fundamentally different kinds of item, and an order also carries non-product charges that behave like items when priced but are not items at all; your design must hold them all together, price them uniformly where they are alike and differently where they are not, and stay open to new kinds of item, add-on, and payment without rewriting the pricing logic.

Design note. This specification states the data, the rules, and the public operations each part must offer. It deliberately does *not* say which of your types should be classes and which interfaces, how inheritance is arranged, or what private helpers you need — deriving those is the assessed part of the work, graded by reading your code and in the viva, not only by passing numbers. Read the companion handout *Interfaces in Object-Oriented Programming: A Beginner's Guide* before you begin.

1. Catalogue Items

Every item, regardless of kind, has: a **SKU**, a **title**, a **unit price**, a live **stock count**, and an owning **seller**. Every item can report these, report how many remain, and **reserve** a quantity — lowering its stock, and refusing when too few remain. This behaviour is identical for all kinds.

There are three kinds, differing only in how they take part in the money:

- **Stocked good** — a warehoused physical product. Has weight (so it ships), standard VAT, may be returned and insured, modest commission.
- **Digital good** — delivered by download. No weight, never ships, lower VAT, cannot be returned or insured, highest commission.
- **Fresh good** — perishable. Has weight and needs a refrigerated (cold-chain) surcharge, VAT-exempt, may be returned and insured, lowest commission.

Each kind states its own **per-unit VAT** and the **commission** the platform earns on it. Rates are fixed in §8. Adding a fourth kind later must not require changes to pricing, delivery, or settlement.

2. Add-on Lines

On top of the products, a customer may add paid **add-ons** that no seller lists and that are not catalogue items: **gift wrapping** (flat charge), **express handling** (larger flat charge), and an **extended warranty plan** (charged as a percentage of the unit price of the one item it protects). Each add-on still sits on the order as its own line with its own charge, its own VAT, and a short receipt label — exactly like a product line. Add-ons earn no seller commission; their charges belong to the platform.

3. Optional Line Capabilities

Every line (product or add-on) can state its charge and its VAT. Beyond that, only *some* lines have the following capabilities; the rest neither have them nor should be asked for them:

Line	Weight	Cold-chain	Insurable	Returnable
Stocked good	yes	no	yes	yes
Digital good	no	no	no	no
Fresh good	yes	yes	yes	yes
Any add-on	no	no	no	no

- **Weight** → delivery. The delivery calculator totals the weight of *only the lines that have a weight*.
- **Cold-chain** → a surcharge added for each fresh line.
- **Insurable** → the customer may insure such a line for a fee; insuring a line that is not insurable is refused.
- **Returnable** → such a line may be returned within its window; returning a non-returnable line, or one past its window, is refused.

4. Order Pricing

An order is priced in this fixed sequence (rates in §8):

1. **Subtotal** — sum of every line's charge: for a product, unit price × quantity; for an add-on, its own charge.
2. **Discount** — if a valid coupon is applied, a percentage off the *discountable* part of the subtotal (stocked goods only), capped at a maximum. An invalid coupon is a refusal, not a silent zero.
3. **Delivery** — shipping for the zone computed from the total billed weight of weighable lines, plus a cold-chain surcharge per fresh line. Zero if the order has no weighable lines.
4. **Tax** — each line's VAT (per-unit VAT rounded up, then × quantity), summed over the order.
5. **Insurance** — for each line the customer chose to insure, its insurance fee.
6. **Service fee** — a small percentage of the subtotal, capped.
7. **Grand total** = subtotal − discount + delivery + tax + insurance + service fee.

A **coupon** has a code, a percentage, a cap, a minimum spend, and a last valid day. Given a discountable base and the current day, it returns the discount, or refuses if it has expired or the order is below its minimum. The same grand total is needed three times — for a *quote*, during *placement*, and again in *settlement* — and must be computed the same way each time.

5. Payment

A placed order is paid in exactly one of three interchangeable ways. The order hands the chosen way a single amount and asks it to **authorise** that amount:

- **Mobile wallet** — debits the wallet immediately; refused if the balance is too low.
- **Card** — succeeds unless the amount exceeds the card's remaining limit, in which case refused.
- **Cash on delivery** — moves no money now, but refused if the grand total is above the COD ceiling.

A **wallet** reports its balance and can be debited (refused below zero) and credited (positive amounts only); its balance cannot be set directly from outside.

6. Placing an Order and Nightly Settlement

Placement is atomic. Placing an order either fully succeeds — every line's stock reserved, payment taken, order recorded as placed — or changes nothing. Every reason it could be refused (insufficient stock on any line, invalid coupon, declined payment) must be checked *before* any stock is reserved or any money moves, so that a refusal leaves every stock count and balance exactly as it was.

Settlement. At day's end, for each seller and over every order placed that day, the run adds the value of that seller's product lines, subtracts the platform commission on each (rate by item kind), subtracts any refunds for returns that seller took back, and reports the remainder as that seller's **payout**. Add-on charges, delivery, tax, and the service fee are **platform revenue**, not any seller's. The run reports every seller's payout and the platform's own revenue for the day.

7. Failure Handling

Two distinct kinds of failure, handled two different ways:

- **Malformed request** — data that should never have been constructed: null SKU/title/seller, negative price, negative stock, quantity ≤ 0 , negative coupon percentage. Reject at construction with an **unchecked** `IllegalArgumentException`; such an object must not come into existence.
- **Business refusal** — a well-formed request the business declines: out of stock, coupon rejected, payment declined, COD over ceiling, non-returnable return. Model these as a family of **checked** exceptions under one common parent, thrown while an order is priced, placed, or returned, and caught by the caller. Payment declines have their own sub-kinds (empty wallet, card over limit, COD over ceiling).

Refusals must be *thrown*, never signalled by a return code, a printed message, or half-completing the work.

8. Rates and a Worked Example

All money is a whole number of **Taka** held in a long; counts are int. Every figure that comes from a percentage is **rounded up to the next whole Taka** (ceiling). VAT is charged *per unit* (round the per-unit VAT up, then multiply by quantity).

Line kind	VAT	Commission / charge
Stocked good	7.5%	commission 8%
Digital good	5%	commission 20%
Fresh good	0%	commission 5%
Gift wrapping	7.5%	flat 50 Taka; no commission
Express handling	7.5%	flat 120 Taka; no commission
Warranty plan	15%	10% of covered item's unit price; no commission

Other fixed figures:

- **Delivery** — inside Dhaka: 60 base + 20 per billed kg; outside Dhaka: 120 base + 35 per billed kg. Billed kg = total weight of weighable lines rounded *up* to the next whole kilogram.
- **Cold-chain** — 50 Taka per fresh line.
- **Discountable** — stocked goods only (digital, fresh, and add-ons never discounted).
- **Service fee** — 1% of subtotal, capped at 100 Taka.
- **Insurance** — 1% of the line's insurable value (unit price $\times$ quantity), minimum 20 Taka.
- **COD ceiling** — 15 000 Taka.

Worked example

All amounts in Taka. Zone: inside Dhaka. Coupon "EID10": 10%, cap 200, minimum spend 1 000, valid through day 150; today is day 100. Lines: *Table Lamp* (stocked, unit 1 200, qty 2, 1 500 g each); *E-book* (digital, unit 300, qty 1); *Hilsa* (fresh, unit 1 600, qty 1, 1 000 g); *Gift wrapping*.

- **Subtotal** = $2(1200) + 300 + 1600 + 50 = 4\,350$.
- **Discount** — discountable base is the stocked line, 2 400; $10\% = 240$, capped $\Rightarrow 200$.
- **VAT** — lamp $\text{ceil}(1200 \times 7.5\%) = 90$, $\times 2 = 180$; e-book $\text{ceil}(300 \times 5\%) = 15$; hilsa 0; gift wrap $\text{ceil}(50 \times 7.5\%) = \text{ceil}(3.75) = 4$. Total = 199.
- **Delivery** — weighable: lamp $1500 \times 2 = 3000$ g + hilsa 1000 g = 4000 g $\rightarrow$ 4 billed kg; shipping = $60 + 4(20) = 140$; cold-chain = 50; delivery = 190.
- **Service fee** — $\text{ceil}(4350 \times 1\%) = \text{ceil}(43.5) = 44$ (under cap).
- **Grand total** = $4350 - 200 + 190 + 199 + 44 = 4\,583$ Taka.

9. Public Operations

The behaviours your design must offer, *by name and effect only*. We give no method signatures or private helpers, and do not say which type is a class and which an interface — `KenaKataTest.java` pins the signatures it calls; wherever two operations repeat a calculation, introduce the private helper yourself.

- **Catalogue item** (any kind) — report SKU, title, unit price, seller, stock remaining; reserve a quantity (lowering stock, refusing when too few); state per-unit charge, per-unit VAT, and commission. Only some kinds also state a per-unit weight, cold-chain surcharge, insurable value, and return window (§3).
- **Add-ons** (gift wrap, express handling, warranty) — state their own charge, per-unit VAT, and receipt label; the warranty computes its charge from the item it protects.
- **Order** — add a product line (item + quantity); add an add-on; apply a coupon; mark a line to insure; produce a *quote* (full breakdown, committing nothing); be *placed* (reserve stock and take payment atomically, changing nothing if refused); accept a *return* of a line within its window; report each price component and its lines.
- **Coupon** — report its code; compute the discount for a discountable base on a given day, throwing if expired or below minimum spend.
- **Delivery calculator** — compute the delivery fee for a set of lines in a zone (billed-weight shipping plus cold-chain for fresh lines).
- **Payment method** (wallet, card, COD) — authorise an amount, moving money if it can and throwing the right payment refusal if not. A **wallet** also reports its balance and can be debited/credited, its balance sealed from outside.
- **Marketplace / settlement** — register sellers; record a placed order; run the day's settlement, reporting each seller's payout and the platform revenue.

10. Testing and main

`KenaKataTest.java` (do not modify it) checks *numbers and failures only*, never structure. It verifies each item kind's charge, VAT, commission, and reservation; add-on charges (including a warranty from its covered item); full orders totalling to the taka (the §8 example and others); delivery over weighable lines only; coupon rules and throws; atomic placement (stock and balances unchanged after a refusal); each failure throwing its own checked kind under the common parent; insuring/returning respecting the capabilities; per-seller settlement and platform revenue; and one payment variable used as wallet, card, then COD.

Your `Main` builds a few sellers and a catalogue across all three kinds, assembles two or three mixed orders (products, add-ons, a coupon, an insured line), prints a quote for each, places them against different payment methods — one deliberately refused, caught, and reported without crashing — processes one return, then runs the nightly settlement.

11. Submission and Ground Rules

- Java 17. Money as whole **Taka** in long, counts in int; round every percentage result up to the next whole Taka (ceiling).
- You may use `java.util.ArrayList` (or similar) for the catalogue, an order's lines, the batch of orders, and the settlement report. The container is not the point; the order must hold every kind of line under one common type, and the settlement every placed order the same way.
- Every rate, cap, and ceiling is fixed in the test file; reproduce them exactly.
- Organise your classes into **logical Java packages** rather than dropping every file into one directory — group types by responsibility (for example, the item family, the payment methods, the exception family, the order/pricing types, and the settlement types each in their own package). Keeping related classes together, and hiding what one package need not expose to another, is itself part of the design you are marked on.

- Partial credit applies — submit whatever compiles. Submit your `.java` sources and a short `README.txt` naming your team and giving one sentence per type stating whether it is a class or an interface and why.